สคริปต์ Demo — ระบบจองคิวและนัดหมายออนไลน์

- **[วงเล็บเหลี่ยม]** = สิ่งที่ต้องกด/พิมพ์ · ข้อความใน `>` = คำพูด (ปรับเป็นภาษาตัวเองได้ เติม ครับ/ค่ะ ตามสะดวก)
- เวลาโดยรวม ~12–15 นาที (คนละ 2–3 นาที)
- **ต้อง demo ก่อน 23:00** (หลังจากนั้นไม่มี slot ว่างของวันนี้ให้จองคิว)
- ใน Postman ให้กด **Send ทีละ request ตามลำดับ** (request หลังใช้ id จาก request ก่อน)

เตรียมก่อนเริ่ม (5 นาทีก่อนนำเสนอ)

เปิด PowerShell ในโฟลเดอร์โปรเจกต์ (ต้องมีไฟล์ `.env` แล้ว ถ้ายังไม่มีให้รัน `copy .env.example .env` ก่อน) แล้วรันทีละบรรทัด:

```
docker compose down -v
docker compose up -d --build
docker compose ps
chcp 65001
```

- `down -v` ล้าง DB ให้ id เริ่มจาก 1 · `ps` ต้องเห็นทุก service เป็น `Up` · `chcp 65001` ให้ภาษาไทยไม่เป็น `????`
- เปิด Postman และ import `postman/queue-booking.postman_collection.json` (เวอร์ชันที่มีชุด `[คิววันนี้]`)
- ขยายตัวอักษร PowerShell ด้วย **Ctrl + ลูกกลิ้งเมาส์**

🎤 คนที่ 1 — เปิด + Auth & User Management (port 8081)

เปิดการนำเสนอ

> สวัสดี วันนี้กลุ่มเราจะนำเสนอ **ระบบจองคิวและนัดหมายออนไลน์** ที่ออกแบบเป็น **Microservices** แบ่งเป็น 4 service ตาม module แต่ละคนรับผิดชอบคนละตัว คือ Auth, Service Management, Booking และ Notification & Report เขียนด้วย **Go** ใช้ **Gin** เป็น HTTP framework, **GORM** ต่อ **PostgreSQL** และรันทั้งหมดด้วย **Docker Compose**

[รัน] `docker compose ps`

> มี 4 service รันอยู่ที่ port 8081 ถึง 8084 และมี Postgres หนึ่งตัว

[รัน] `docker compose exec postgres psql -U postgres -c "\l"`

> เราใช้หลัก **Database-per-Service** — ใช้ Postgres container เดียว แต่แยกเป็น 4 database แต่ละ database มี user เป็นเจ้าของคนละคน

[รัน] `docker compose exec postgres sh -c 'PGPASSWORD=$AUTH_DB_PASSWORD psql -h localhost -U auth_user -d booking_db -l'`

> ถ้า auth-service พยายามเข้า database ของ booking จะขึ้น **permission denied** (สีแดง = ถูกต้อง) แปลว่าข้อมูลแยกกันจริง ถ้าต้องการข้อมูลของ service อื่นต้องเรียกผ่าน API เท่านั้น

Auth & User Management

> ส่วนแรกคือ **auth-service** ดูแลการสมัครสมาชิก login และสิทธิ์การใช้งาน

[กด Register customer]

> สมัครสมาชิกเป็นลูกค้า ได้ 201 Created พร้อมข้อมูล user

[กด Register provider]

> สมัครอีก role คือผู้ให้บริการ

[กด Register duplicate email (409)]

> ถ้าใช้อีเมลซ้ำ ระบบตอบ **409 Conflict**

[กด Login customer]

> login แล้วได้ **JWT token** อายุ 3600 วินาที (1 ชั่วโมง) Postman เก็บ token ไว้ใช้กับ request ถัดไปให้อัตโนมัติ

[กด Login provider → Login admin → Login wrong password (401)]

> ถ้ารหัสผ่านผิดได้ **401 Unauthorized**

[กด Get my profile → Update my profile]

> ใช้ token ดูและแก้โปรไฟล์ของตัวเองได้

[กด List users (admin)]

> admin ดูรายชื่อผู้ใช้ทั้งหมดได้ กรองตาม role และแบ่งหน้าได้

[กด List users as customer (403)]

> ถ้าลูกค้าเรียก endpoint เดียวกันได้ **403 Forbidden** เพราะระบบตรวจสิทธิ์ตาม role

[กด Suspend user → Activate user]

> admin ระงับบัญชีแล้วเปิดคืนได้ ระหว่างถูกระงับ token เดิมของ user ใช้ไม่ได้ทันที

[รัน] `docker compose exec postgres psql -U auth_user -d auth_db -c "select id, name, email, role, status, left(password_hash, 20) as password_hash from users"`

> รหัสผ่านไม่ได้เก็บเป็นข้อความตรงๆ แต่เข้ารหัสด้วย **bcrypt** (ขึ้นต้นด้วย `$2a$`) ต่อไปขอส่งให้คนที่ 2 นำเสนอ Service Management

🎤 คนที่ 2 — Service Management (port 8082)

> ส่วนนี้คือ **service-service** ดูแลรายการบริการที่เปิดให้จอง และช่วงเวลาที่แต่ละบริการเปิดรับ

[กด Create service (provider)]

> ผู้ให้บริการสร้างบริการ "ตัดผมชาย" 30 นาที 150 บาท ได้ 201 Created

[กด Create service as customer (403)]

> ลูกค้าสร้างบริการไม่ได้ ได้ **403** — เฉพาะ provider หรือ admin

[กด List services → Get service]

> ดูรายการบริการเป็น **สาธารณะ** ไม่ต้อง login ค้นหาด้วยหมวดหมู่หรือคำค้นได้ และหน้ารายละเอียดบอกว่าเป็นของผู้ให้บริการคนไหน

[กด Update service]

> แก้ได้เฉพาะ **เจ้าของบริการหรือ admin** ถ้า provider คนอื่นแก้จะได้ 403

[กด Add availability (tomorrow 09:00-12:00)]

> กำหนดช่วงเวลาเปิดรับจอง เลือกวันในสัปดาห์ เวลาเริ่ม และเวลาสิ้นสุด ตัวอย่างนี้คือพรุ่งนี้ 9 โมงถึงเที่ยง

[กด Get available slots]

> ระบบ **คำนวณ slot ให้อัตโนมัติ** โดยแบ่งช่วงเวลาตามระยะเวลาของบริการ บริการนี้ 30 นาที เลยได้ 9:00, 9:30, 10:00 ... ข้อมูลนี้ booking-service ใช้ตรวจสอบตอนลูกค้าจอง

[รัน] `docker compose exec postgres psql -U service_user -d service_db -c "select * from services" -c "select * from availability"`

> ตาราง availability มี foreign key ไปที่ services แบบ **ON DELETE CASCADE** ลบบริการแล้วช่วงเวลาถูกลบตามไปด้วย ต่อไปขอส่งให้คนที่ 3 นำเสนอการจองและคิว

🎤 คนที่ 3 — Booking & Realtime Queue (port 8083)

> ส่วนนี้คือ **booking-service** หัวใจของระบบ ดูแลการจองนัดหมายและสถานะคิวแบบเรียลไทม์

การจอง

[กด Create booking]

> ลูกค้าจองตาม slot ที่ว่าง ได้สถานะ **pending** booking-service **ไม่ได้อ่านฐานข้อมูลของ service-service** แต่เรียก API ของ service-service เพื่อเช็กว่าบริการมีจริงและเวลาที่จองอยู่ในช่วงที่เปิดรับ

[กด Create booking same slot (409)]

> จอง slot เดียวกันซ้ำได้ **409 Conflict** (คิวเต็ม) นอกจากเช็กในโค้ดแล้วยังมี **unique index** ใน database ถึงสองคนกดพร้อมกันก็สำเร็จได้คนเดียว

[กด List my bookings → Get booking]

> ลูกค้าเห็นเฉพาะการจองของตัวเอง ส่วน provider เห็นการจองของบริการที่ตัวเองดูแล

[กด Reschedule booking]

> เลื่อนนัดไป slot อื่นได้ ถ้า slot ใหม่มีคนจองแล้วได้ 409

[ข้าม "Get realtime queue" ไปเลย — ยังไม่มีคิววันนี้ จะไปโชว์ในส่วนถัดไป]

[กด Cancel booking]

> ยกเลิกได้ และ slot ที่ยกเลิกแล้วว่างให้คนอื่นจองต่อทันที

[กด Create 2nd booking → Update booking status (provider)]

> ผู้ให้บริการอัปเดตสถานะงานได้ เป็น in_progress, completed หรือ no_show

คิวเรียลไทม์ ⭐

> ต่อไปเป็นไฮไลต์ คือ **สถานะคิวแบบเรียลไทม์** จะจำลองสถานการณ์ในร้านวันนี้

[กด [คิววันนี้] Add availability for today → Get today slots]

> เปิดรับจองวันนี้ แล้วระบบเลือก slot ถัดไปที่ยังไม่เลยเวลามาให้

[กด [คิววันนี้] Book today - queue #1 → queue #2]

> มีลูกค้าจองคิววันนี้ 2 คิว

[กด [คิววันนี้] Queue - 2 waiting]

> ตอนนี้ `current_serving` เป็น null ยังไม่มีใครรับบริการ และมี 2 คิวรออยู่ใน `waitin g` เรียงตามเวลานัด

[กด [คิววันนี้] Provider starts serving #1]

> ร้านเรียกคิวที่ 1

[กด [คิววันนี้] Queue - serving #1]

> **คิวขยับแล้ว** — คิวที่ 1 กำลังรับบริการ เหลือคิวที่ 2 รอ

[กด [คิววันนี้] Provider completes #1 → Provider starts serving #2 → Queue - serving #2]

> คิวที่ 1 เสร็จ เรียกคิวที่ 2 ตอนนี้คิวที่ 2 กำลังรับบริการ และไม่มีคิวรอแล้ว API ดูคิวนี้ **ไม่ต้อง login** เอาไปขึ้นจอหน้าร้านได้ หน้าจอเรียก API นี้ซ้ำทุกไม่กี่วินาทีก็เห็นคิวอัปเดตตลอด

[รัน] `docker compose exec postgres psql -U booking_user -d booking_db -c "select id, service_id, date, time, status from bookings order by id" -c "select * from queue_status"`

> ตาราง `queue_status` เก็บว่าแต่ละบริการกำลังให้บริการคิวไหน ตรงกับที่ API แสดง ต่อไปขอส่งให้คนที่ 4 นำเสนอ Notification และ Report

🎤 คนที่ 4 — Notification & Report (port 8084)

> ส่วนสุดท้ายคือ **notification-service** ดูแลการแจ้งเตือนผู้ใช้และรายงานสำหรับ admin

Notification

[กด Send notification (internal)]

> endpoint นี้เป็น **endpoint ภายในระบบ** ใช้ส่งแจ้งเตือนก่อนถึงเวลานัด ผู้ใช้ทั่วไปเรียกไม่ได้ ต้องแนบ key ภายใน `X-Internal-Key` ตอนส่ง notification-service ดึงข้อมูลการจองจาก booking-service ผ่าน API เพื่อรู้ว่าต้องแจ้งใคร นัดวันไหนเวลาไหน

[กด List my notifications]

> ลูกค้าเห็นแจ้งเตือนของตัวเอง กรองเฉพาะอันที่ยังไม่อ่านได้

[กด Mark notification read]

> ทำเครื่องหมายว่าอ่านแล้ว ถ้าเป็นแจ้งเตือนของคนอื่นได้ 404

Report

[กด Report summary (admin)]

> admin ดูสรุปจำนวนการจองในช่วงวันที่เลือก แยกตามสถานะ เสร็จแล้ว ยกเลิก หรือไม่มาตามนัด ข้อมูลการจองอยู่ใน booking-service ตัว report จึงดึงผ่าน API ของ booking-service มาคำนวณ ไม่ได้อ่านฐานข้อมูลของ booking โดยตรง

[กด Report summary as customer (403)]

> ลูกค้าดูรายงานไม่ได้ ได้ 403

[กด Cancellation rate by service → Cancellation rate all services]

> ดูอัตราการยกเลิกได้ทั้งแบบบริการเดียว และทุกบริการพร้อมกัน

[ที่ Export CSV กดลูกศรข้างปุ่ม Send → Send and Download]

> **ส่งออกรายงานเป็นไฟล์ CSV** เปิดใน Excel ได้เลย

[รัน] `docker compose exec postgres psql -U notification_user -d notificatio n_db -c "select * from notifications" -c "select * from report_logs"`

[รัน] `docker compose exec notification-service ls -l /data/reports`

> ทุกครั้งที่สร้างรายงาน ระบบบันทึกประวัติไว้ใน `report_logs` ว่า admin คนไหนสร้าง และไฟล์ CSV ที่ export ถูกเก็บไว้จริงตาม path ที่บันทึก ขอส่งกลับให้คนที่ 1 ปิดท้าย

🎤 คนที่ 1 — ปิดท้าย

[กด Delete service]

> ลบบริการได้ 204 No Content

[กด Logout → Get profile after logout (401)]

> หลัง logout เอา token เดิมมาใช้ได้ 401 ทันที ถึง token ยังไม่หมดอายุ เพราะระบบ **revoke token** จริง service อื่นทุกตัวตรวจ token โดยเรียก auth-service การ logout หรือระงับบัญชีจึงมีผลกับ **ทุก service พร้อมกัน** สรุป ระบบมี 4 microservices แต่ละตัวมีฐานข้อมูลของตัวเอง คุยกันผ่าน API เท่านั้น และทำครบทุก path ตาม spec ทั้ง 27 path ขอบคุณ มีคำถามเชิญได้เลย

หลังจบ: `docker compose stop`

❓ คำถามที่อาจโดนถาม

ใคร	คำถาม	คำตอบ
คนที่ 1	ทำไมใช้ Postgres container เดียว	ประหยัดทรัพยากร แต่แยก database และ user ต่างหาก จึงยังเป็น Database-per-Service จริง จะแยกเป็น 4 container ก็แก้แค่ docker-compose ไม่ต้องแก้โค้ด
คนที่ 1	token ที่ logout แล้วเก็บไว้ที่ไหน	denylist ใน memory ของ auth-service ข้อจำกัดคือ restart แล้วหาย แต่ token อายุแค่ 1 ชั่วโมง ถ้าใช้จริงควรย้ายไป Redis
คนที่ 1	service อื่นรู้ได้ยังไงว่า user คือใคร	ส่ง token ไปถาม auth-service ที่ `GET /api/users/me` ทุกครั้ง
คนที่ 2	slot ที่ได้หักคิวที่ถูกจองแล้วหรือยัง	ยัง แสดงตามตารางเวลา ส่วนการกันจองซ้ำเป็นหน้าที่ของ booking-service (ตอบ 409)
คนที่ 2	ลบบริการแล้ว availability ไปไหน	ถูกลบตามด้วย foreign key แบบ ON DELETE CASCADE
คนที่ 3	สองคนกดจองพร้อมกันเป๊ะ	มี partial unique index บน service_id, date, time (เฉพาะที่ยังไม่ยกเลิก) database ให้สำเร็จได้คนเดียว อีกคนได้ 409
คนที่ 3	"เรียลไทม์" ใช้ WebSocket ไหม	ยังไม่ใช้ เป็น REST ให้หน้าจอเรียกซ้ำเป็นระยะ (polling) ถ้าทำต่อเพิ่ม WebSocket หรือ Server-Sent Events ได้
คนที่ 3	ยกเลิก/เลื่อนนัดได้ตอนไหน	เฉพาะสถานะ pending ถ้าเริ่มให้บริการแล้วได้ 409
คนที่ 4	ส่ง email จริงไหม	ยังไม่ส่งจริง บันทึกลงฐานข้อมูลและเขียน log ถ้าจะส่งจริงต้องต่อผู้ให้บริการอีเมล และมี scheduler มาเรียก endpoint send
คนที่ 4	report join ข้อมูลข้าม database ยังไง	ไม่ได้ join ข้าม database แต่ดึงผ่าน API ของ booking-service ด้วย token ของ admin แล้วคำนวณใน notification-service

ถ้าพลาดระหว่าง demo

อาการ	ทำอะไร
Postman ได้ error แปลกๆ / กดข้ามลำดับ	เริ่มใหม่ตั้งแต่ **Register customer** (อีเมลสุ่มใหม่ทุกครั้ง ไม่ต้องล้าง DB)
service ไม่ขึ้น / ขึ้น `Restarting`	`docker compose logs <ชื่อ service> --tail 30`
`[คิววันนี้] Get today slots` fail	เลย 23:00 แล้ว ไม่มี slot ของวันนี้เหลือ — อธิบายด้วยคำพูดแทน